

LAB-10

Page replacement algorithms:

```
#include <stdio.h>
```

```
int findPage(int frames[], int n, int page) {  
    for (int i = 0; i < n; i++) {  
        if (frames[i] == page)  
            return 1;  
    }  
    return 0;  
}
```

```
void displayFrames(int frames[], int n) {  
    for (int i = 0; i < n; i++) {  
        if (frames[i] == -1)  
            printf("- ");  
        else  
            printf("%d ", frames[i]);  
    }  
    printf("\n");  
}
```

```
void FIFO(int pages[], int p, int f) {  
    int frames[10], front = 0, faults = 0;  
  
    for (int i = 0; i < f; i++)  
        frames[i] = -1;  
  
    printf("\nFIFO Page Replacement:\n");  
  
    for (int i = 0; i < p; i++) {  
        if (!findPage(frames, f, pages[i])) {  
            frames[front] = pages[i];  
            front = (front + 1) % f;  
            faults++;  
  
            printf("Page %d -> ", pages[i]);  
            displayFrames(frames, f);  
        }  
    }  
}
```

```

    } else {
        printf("Page %d -> No Page Fault\n", pages[i]);
    }
}

printf("Total Page Faults in FIFO = %d\n", faults);
}

```

```

void Optimal(int pages[], int p, int f) {
    int frames[10], faults = 0;

    for (int i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nOptimal Page Replacement:\n");

    for (int i = 0; i < p; i++) {
        if (!findPage(frames, f, pages[i])) {
            int empty = -1;

            for (int j = 0; j < f; j++) {
                if (frames[j] == -1) {
                    empty = j;
                    break;
                }
            }

            if (empty != -1) {
                frames[empty] = pages[i];
            } else {
                int farthest = -1, replaceIndex = -1;

                for (int j = 0; j < f; j++) {
                    int k;
                    for (k = i + 1; k < p; k++) {
                        if (frames[j] == pages[k])
                            break;
                    }

                    if (k == p) {
                        replaceIndex = j;
                    }
                }
            }
        }
    }
}

```

```

        break;
    }

    if (k > farthest) {
        farthest = k;
        replaceIndex = j;
    }
}

frames[replaceIndex] = pages[i];
}

faults++;

printf("Page %d -> ", pages[i]);
displayFrames(frames, f);
} else {
    printf("Page %d -> No Page Fault\n", pages[i]);
}
}

printf("Total Page Faults in Optimal = %d\n", faults);
}

void LRU(int pages[], int p, int f) {
    int frames[10], recent[10], faults = 0, time = 0;

    for (int i = 0; i < f; i++) {
        frames[i] = -1;
        recent[i] = 0;
    }

    printf("\nLRU Page Replacement:\n");

    for (int i = 0; i < p; i++) {
        time++;

        int found = 0;

        for (int j = 0; j < f; j++) {
            if (frames[j] == pages[i]) {

```

```

        found = 1;
        recent[j] = time;
        break;
    }
}

if (!found) {
    int empty = -1;

    for (int j = 0; j < f; j++) {
        if (frames[j] == -1) {
            empty = j;
            break;
        }
    }

    if (empty != -1) {
        frames[empty] = pages[i];
        recent[empty] = time;
    } else {
        int min = recent[0], replaceIndex = 0;

        for (int j = 1; j < f; j++) {
            if (recent[j] < min) {
                min = recent[j];
                replaceIndex = j;
            }
        }

        frames[replaceIndex] = pages[i];
        recent[replaceIndex] = time;
    }

    faults++;

    printf("Page %d -> ", pages[i]);
    displayFrames(frames, f);
} else {
    printf("Page %d -> No Page Fault\n", pages[i]);
}
}

```

```

    printf("Total Page Faults in LRU = %d\n", faults);
}

int main() {
    int pages[50], p, f;

    printf("Enter number of pages: ");
    scanf("%d", &p);

    printf("Enter page reference string:\n");
    for (int i = 0; i < p; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &f);

    FIFO(pages, p, f);
    Optimal(pages, p, f);
    LRU(pages, p, f);

    return 0;
}

```

Output:

```

Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

```

```

FIFO Page Replacement:
Page 7 -> 7 - -
Page 0 -> 7 0 -
Page 1 -> 7 0 1
Page 2 -> 2 0 1
Page 0 -> No Page Fault
Page 3 -> 2 3 1
Page 0 -> 2 3 0
Page 4 -> 4 3 0
Page 2 -> 4 2 0
Page 3 -> 4 2 3
Page 0 -> 0 2 3
Page 3 -> No Page Fault
Total Page Faults in FIFO = 10

```

```
Optimal Page Replacement:
Page 7 -> 7 - -
Page 0 -> 7 0 -
Page 1 -> 7 0 1
Page 2 -> 2 0 1
Page 0 -> No Page Fault
Page 3 -> 2 0 3
Page 0 -> No Page Fault
Page 4 -> 2 4 3
Page 2 -> No Page Fault
Page 3 -> No Page Fault
Page 0 -> 0 4 3
Page 3 -> No Page Fault
Total Page Faults in Optimal = 7
```

```
LRU Page Replacement:
Page 7 -> 7 - -
Page 0 -> 7 0 -
Page 1 -> 7 0 1
Page 2 -> 2 0 1
Page 0 -> No Page Fault
Page 3 -> 2 0 3
Page 0 -> No Page Fault
Page 4 -> 4 0 3
Page 2 -> 4 0 2
Page 3 -> 4 3 2
Page 0 -> 0 3 2
Page 3 -> No Page Fault
Total Page Faults in LRU = 9
PS C:\MinGW\bin> █
```

Observation:

LAB-10

Page replacement Algorithm:

```
#include <stdio.h>

int findPage (int frames[], int n, int page) {
    for (int i = 0; i < n; i++) {
        if (frames[i] == page)
            return 1;
    }
    return 0;
}

void displayFrames (int frames[], int n) {
    for (int i = 0; i < n; i++) {
        if (frames[i] == -1)
            printf ("-");
        else
            printf ("%d", frames[i]);
    }
    printf ("\n");
}

void FIFO (int pages[], int p, int f) {
    int frames[10], front = 0, faults = 0;
    for (int i = 0; i < p; i++) {
        if (!findPage (frames, f, pages[i])) {
            printf ("In FIFO Page Replacement : \n");
            frames[front] = pages[i];
            front = (front + 1) % f;
            faults++;
            printf ("Page %d -> ", pages[i]);
            displayFrames (frames, f);
        }
        else {
            printf ("Page %d -> No Page Fault \n", pages[i]);
        }
    }
    printf ("Total page Faults in FIFO = %d \n", faults);
}
```

```

void optimal (int pages[], int P, int f) {
    int frames[10], faults = 0;
    for (int i = 0; i < f; i++) {
        frames[i] = -1;
    }
    printf ("\nOptimal Page Replacement : \n");
    for (int i = 0; i < P; i++) {
        if (!findPage (frames, f, pages[i])) {
            int empty = -1;
            for (int j = 0; j < f; j++) {
                if (frames[j] == -1) {
                    empty = j;
                    break;
                }
            }
            if (empty != -1) {
                frames[empty] = pages[i];
            }
            else {
                int farthest = -1; replaceIndex = -1;
                for (int j = 0; j < f; j++) {
                    int k;
                    for (k = 0; k < P; k++) {
                        if (frames[j] == pages[k])
                            break;
                    }
                    if (k == P) {
                        replaceIndex = j;
                        break;
                    }
                    if (k > farthest) {
                        farthest = k;
                        replaceIndex = j;
                    }
                }
                frames[replaceIndex] = pages[i];
            }
            faults++;
            printf ("Page : %d → ", pages[i]);
            displayFrames (frames, f);
        }
        else {
            printf ("Page : %d → No page Fault \n", pages[i]);
        }
    }
}

```



```

printf ("Total Page Faults in optimal = %d\n", faults);
}

void LRU (int pages[], int P, int f) {
    int frames[f], recent[f], faults = 0, time = 0;

    for (int i = 0; i < f; i++) {
        frames[i] = -1;
        recent[i] = 0;
    }

    printf ("In LRU Page Replacement :\n");

    for (int i = 0; i < P; i++) {
        time++;
        int found = 0;
        for (int j = 0; j < f; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                recent[j] = time;
                break;
            }
        }

        if (!found) {
            int empty = -1;
            for (int j = 0; j < f; j++) {
                if (frames[j] == -1) {
                    break;
                }
            }

            if (empty != -1) {
                frames[empty] = pages[i];
                recent[empty] = time;
            } else {
                int min = recent[0], replaceIndex = 0;
                for (int j = 1; j < f; j++) {
                    if (recent[j] < min) {
                        min = recent[j];
                        replaceIndex = j;
                    }
                }

                frames[replaceIndex] = pages[i];
                recent[replaceIndex] = time;
            }
        }
    }
}

```

```

    faults++;
    printf("Page-%d -> ", pages[i]);
    displayFrames(frames, f);
} else {
    printf("Page-%d -> No page fault\n", pages[i]);
}
}
printf("Total page faults in LRU = %d\n", faults);

int main() {
    int pages[50], P, f;
    printf("Enter number of pages: ");
    scanf("%d", &P);
    printf("Enter page reference string: ");
    scanf("%d", &P);
    printf("Enter\n");
    for(int i=0; i<P; i++){
        scanf("%d", &pages[i]);
    }
    printf("Enter number of frames: ");
    scanf("%d", &f);
    FIFO(pages, P, f);
    Optimal(pages, P, f);
    LRU(pages, P, f);
    return 0;
}

```

Output:

```

Enter the number of frames: 3
Enter number of pages, in reference string: 7
Enter the reference string: 1 0 3 0 3 5 6 2

```

--- FIFO Result ---

1 → [1 - -]
3 → [1 3 -]
0 → [1 3 0]
3 → [1 3 0] Hit
5 → [5 3 0]
6 → [5 6 0]
3 → [5 6 3]

Total page faults: 6

--- Optimal Result ---

1 → [1 - -]
3 → [1 3 -]
0 → [1 3 0]
3 → [1 3 0] Hit
5 → [5 3 0]
6 → [6 3 0]
3 → [6 3 0] Hit

Total page faults : 5

--- LRU Result ---

1 → [1 - -]
3 → [1 3 -]
0 → [1 3 0]
3 → [1 3 0] Hit
5 → [5 3 0]
6 → [5 3 6]
3 → [5 3 6] Hit

Total page faults : 5